

Databases for Marketing

Ad Targeting and Surveillance Pricing

UC San Diego

Logistics

- PA 1 due at midnight!
- Some hiccups/bugs with autograder & Gradescope enrollments
 - If you haven't been able to submit yet, come talk to us after class!
- If you are not yet officially enrolled (i.e., on Canvas) use Gradescope entry code!

291I: **7BXE5W**

190: **3ZXR5K**

Previously on CSE 190|291...

- We talked about what the internet is
- We talked about the basics of networking: how **packets** are directed via **protocols** to specific **addresses** and **ports**
- We talked in more detail about the **Hypertext Transfer Protocol (HTTP)** including the structure of **request** and **response** and how it relates to **cookies**

Note C.—Questions Nos. 13, 14, 22 and 23 are not to be asked in respect to persons under 10 years of age.

SCHEDULE 1.—Inhabitants in....., in the County of....., Sta
enumerated by me on the..... day of June, 1880.

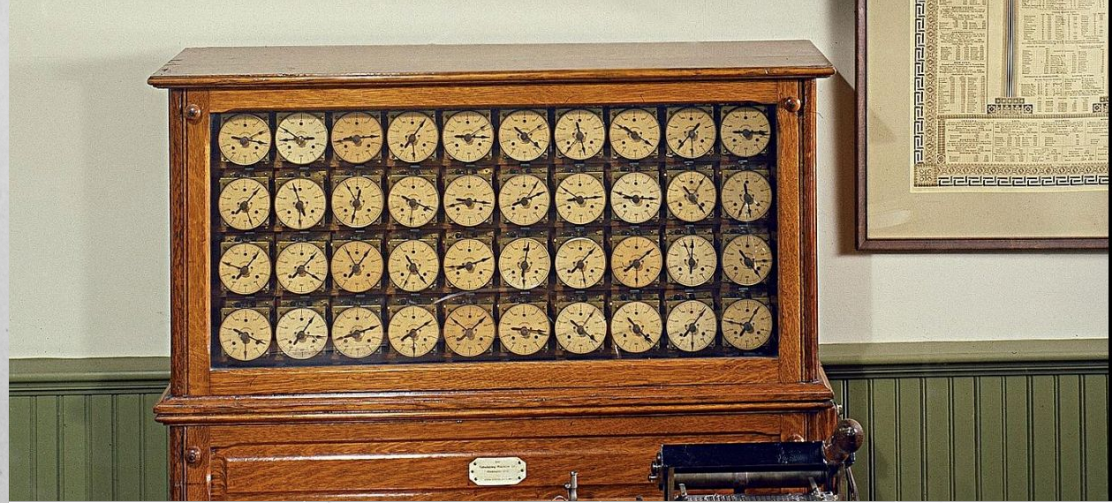
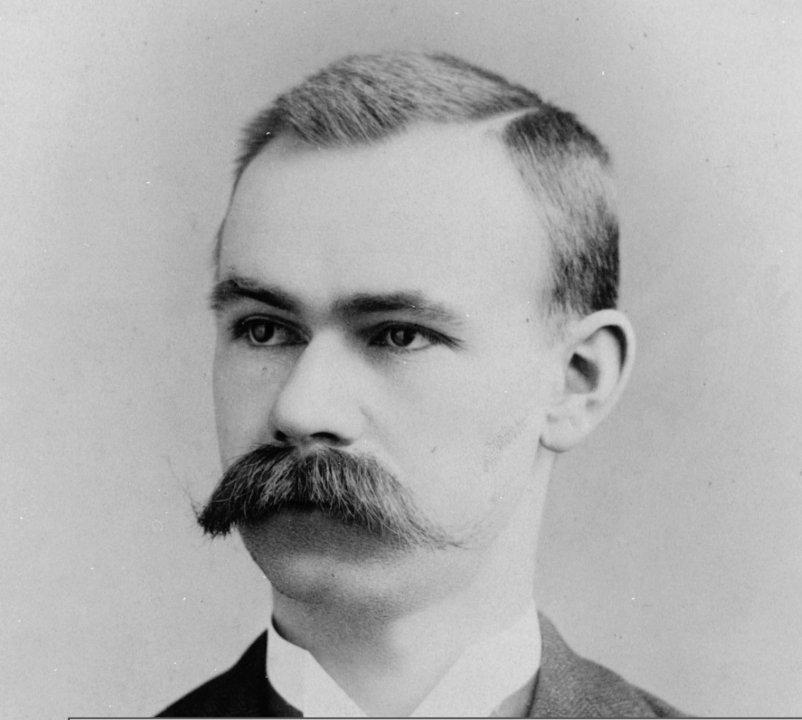
[illegible]

Enumeration Dist. No.

Note C.—Questions Nos. 13, 14, 22 and 23 are not to be asked in respect to persons under 10 years of age.

SCHEDULE 1.—Inhabitants in....., in the County of....., Sta
enumerated by me on the..... day of June, 1880.

1880 U.S. census took 8 years to finish counting



For the 1890 U.S. census, Herman Hollerith invented a new "tabulating machine"

[illegible]

1	2	3	4	CM	UM	Jp	Ch	Oc	In	20	50	80	Dv	Un	3	4	3	4	A	E	L	a	g
5	6	7	8	CL	UL	O	Mu	Qd	Mo	25	55	85	Wd	CY	1	2	1	2	B	F	M	b	h
1	2	3	4	CS	US	Mb	B	M	0	30	60	0	2	Mr	0	15	0	15	C	G	N	c	i
5	6	7	8	No	Hd	Wf	W	F	5	35	65	1	3	Sg	5	10	5	10	D	H	O	d	k
1	2	3	4	Fh	Ff	Fm	7	1	10	40	70	90	4	0	1	3	0	2	St	I	P	e	l
5	6	7	8	Hh	Hf	Hm	8	2	15	45	75	95	100	Un	2	4	1	3	4	K	Un	f	m
1	2	3	4	X	Un	Ft	9	3	i	c	X	R	L	E	A	6	0	US	Ir	Sc	US	Ir	Sc
5	6	7	8	Ot	En	Mt	10	4	k	d	Y	S	M	F	B	10	1	Gr	En	Wa	Gr	En	Wa
1	2	3	4	W	R	OK	11	5	l	e	Z	T	N	G	C	15	2	Sw	FC	EC	Sw	FC	EC
5	6	7	8	7	4	1	12	6	m	f	NG	U	O	H	D	Un	3	Nw	Bo	Hu	Nw	Bo	Hu
1	2	3	4	8	5	2	Oc	0	n	g	a	V	P	I	Al	Na	4	Dk	Fr	It	Dk	Fr	It
5	6	7	8	9	6	3	0	p	o	h	b	W	Q	K	Un	Pa	5	Ru	Ot	Un	Ru	Ot	Un

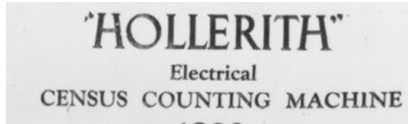
1	2	3	4	CM	UM	Jp	Ch	Oc	In	20	50	80	Dv	Un	3	4	3	4	A	E	L	a	g	
5	6	7	8	CL	UL	O	Mu	Qd	Mo	25	55	85	Wd	CY	1	2	1	2	B	F	M	b	h	
1	2	3	4	CS	US	Mb	B	M	0	30	60	0	2	Mr	0	15	0	15	C	G	N	c	i	
5	6	7	8	No	Hd	Wf	W	F	5	35	65	1	3	Sg	5	10	5	10	D	H	O	d	k	
1	2	3	4	Fh	Ff	Fm	7	1	10	40	70	90	4	0	1	3	0	2	St	I	P	e	l	
5	6	7	8	Hh	Hf	Hm	8	2	15	45	75	95	100	Un	2	4	1	3	4	K	Un	f	m	
1	2	3	4	X	Un	Ft	9	3	i	c	X	R	L	E	A	6	0	US	Ir	Sc	US	Ir	Sc	
5	6	7	8	Ot	En	Mt	10	4	k	d	Y	S	M	F	B	10	1	Gr	En	Wa	Gr	En	Wa	
1	2	3	4	W	R	OK	11	5	l	e	Z	T	N	G	C	15	2	Sw	FC	EC	Sw	FC	EC	
5	6	7	8	7	4	1	12	6	m	f	NG	U	O	H	D	Un	3	Nw	Bo	Hu	Nw	Bo	Hu	
1	2	3	4	8	5	2	Oc	0	n	g	a	V	P	I	Al	Na	4	Dk	Fr	It	Dk	Fr	It	
5	6	7	8	9	6	3	0		p	o	h	b	W	Q	K	Un	Pa	5	Ru	Ot	Un	Ru	Ot	Un

1	2	3	4	CM	UM	Jp	Ch	Oc	In	20	50	80	Dv	Un	3	4	3	4	A	E	L	a	g
5	6	7	8	CL	UL	O	Mu	Qd	Mo	25	55	85	Wd	CY	1	2	1	2	B	F	M	b	h
1	2	3	4	CS	US	Mb	B	M	0	30	60	0	2	Mr	0	15	0	15	C	G	N	c	i
5	6	7	8	No	Hd	Wf	W	F	5	35	65	1	3	Sg	5	10	5	10	D	H	O	d	k
1	2	3	4	Fh	Ff	Fm	7	1	10	40	70	90	4	0	1	3	0	2	St	I	P	e	l
5	6	7	8	Hh	Hf	Hm	8	2	15	45	75	95	100	Un	2	4	1	3	4	K	Un	f	m
1	2	3	4	X	Un	Ft	9	3	i	c	X	R	L	E	A	6	0	US	Ir	Sc	US	Ir	Sc
5	6	7	8	Ot	En	Wa	10	4	k	d	Y	S	M	F	B	10	1	Gr	En	Wa	Gr	En	Wa
1	2	3	4	W	R	OK	11	5	l	e	Z	T	N	G	C	15	2	Sw	FC	EC	Sw	FC	EC
5	6	7	8	7	4	1	12	c	m	f	NG	U	O	H	D	Un	3	Nw	Bo	Hu	Nw	Bo	Hu
1	2	3	4	8	5	2	Oc	0	n	g	a	V	P	I	Al	Na	4	Dk	Fr	It	Dk	Fr	It
5	6	7	8	9	6	3	0	p	o	h	b	W	Q	K	Un	Pa	5	Ru	Ot	Un	Ru	Ot	Un

1	2	3	4	CM	UM	Jp	Ch	Oc	In	20	50	80	Dv	Un	3	4	3	4	A	E	L	a	g
5	6	7	8	CL	UL	O	Mu	Qd	Mo	25	55	85	Wd	CY	1	2	1	2	B	F	M	b	h
1	2	3	4	CS	US	Mb	B	M	0	30	60	0		Mr	0	15	0	15	C		N	c	i
5	6	7	8	No	Hd	Wf	W	F	5	35	65	1		Sg	5	10	5	10	D		O	d	k
1	2	3	4	Fh	Ff	Fm	7	1	10	40	70	90		0	1	3	0	2	St		P	e	l
5	6	7	8	Hh	Hf	Hm	8	2	15	45	75	95	100	Un	2	4	1	3	4	K	Un	f	m
1	2	3	4	X	Un	Ft	9	3	i	c	X	R	L	E	A	6	0	US	Ir	Sc	US	Ir	Sc
5	6	7	8	Ot	En	Mt	10	4	k	d	Y	S	M	F	B	10	1	Gr	En	Wa	Gr	En	Wa
1	2	3	4	W	R	OK	11	5	l	e	Z	T	N	G	C	15	2	Sw	FC	EC	Sw	FC	EC
5	6	7	8	7	4	1	12	6	m	f	NG	U	O	H	D	Un	3	Nw	Bo	Hu	Nw	Bo	Hu
1	2	3	4	8	5	2	Oc	0	n	g	a	V	P	I	Al	Na	4	Dk	Fr	It	Dk	Fr	It
5	6	7	8	9	6	3	0	p	o	h	b	W	Q	K	Un	Pa	5	Ru	Ot	Un	Ru	Ot	Un

1	2	3	4	CM	UM	Op	Ch	Oc	In	20	50	80	Dv	Un	3	4	3	4	A	E	L	a	g
5	6	7	8	CL	UL	O	Mu	Qd	Mo	25	55	85	Wd	CY	1	2	1	2	B	F	M	b	h
1	2	3	4	CS	US	M	B	M	0	30	60	0	2	Mr	0	15	0	15	C	G	N	c	i
5	6	7	8	No	Hd	W	W	F	5	35	65	1	3	Sg	5	10	5	10	D	H	O	d	k
1	2	3	4	Fh	Ff	Fm	7	1	10	40	70	90	4	0	1	3	0	2	St	I	P	e	l
5	6	7	8	Hh	Hf	Hm	8	2	15	45	75	95	100	Un	2	4	1	3	4	K	Un	f	m
1	2	3	4	X	Un	Ft	9	3	i	c	X	R	L	E	A	6	0	US	Ir	Sc	US	Ir	Sc
5	6	7	8	Ot	En	Mt	10	4	k	d	Y	S	M	F	B	10	1	Gr	En	Wa	Gr	En	Wa
1	2	3	4	W	R	OK	11	5	l	e	Z	T	N	G	C	15	2	Sw	FC	EC	Sw	FC	EC
5	6	7	8	7	4	1	12	6	m	f	NG	U	O	H	D	Un	3	Nw	Bo	Hu	Nw	Bo	Hu
1	2	3	4	8	5	2	Oc	0	n	g	a	V	P	I	Al	Na	4	Dk	Fr	It	Dk	Fr	It
5	6	7	8	9	6	3	0	p	o	h	b	W	Q	K	Un	Pa	5	Ru	Ot	Un	Ru	Ot	Un

Tabulating Machines



1889

Hollerith conducted business under his own name



1896

Tabulating Machine Company founded



1911

TMC+3 companies merged →
Computing-
Tabulating-
Recording Company



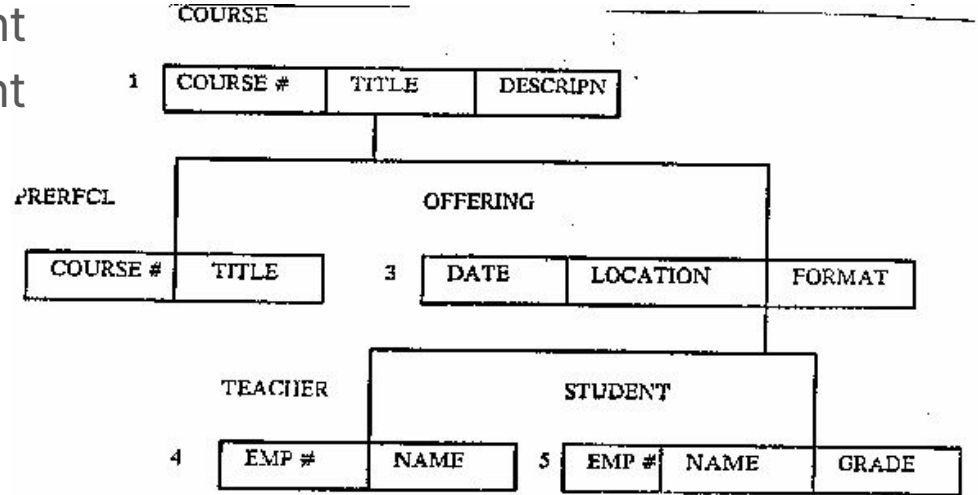
1924

Renamed IBM

Hierarchical Databases

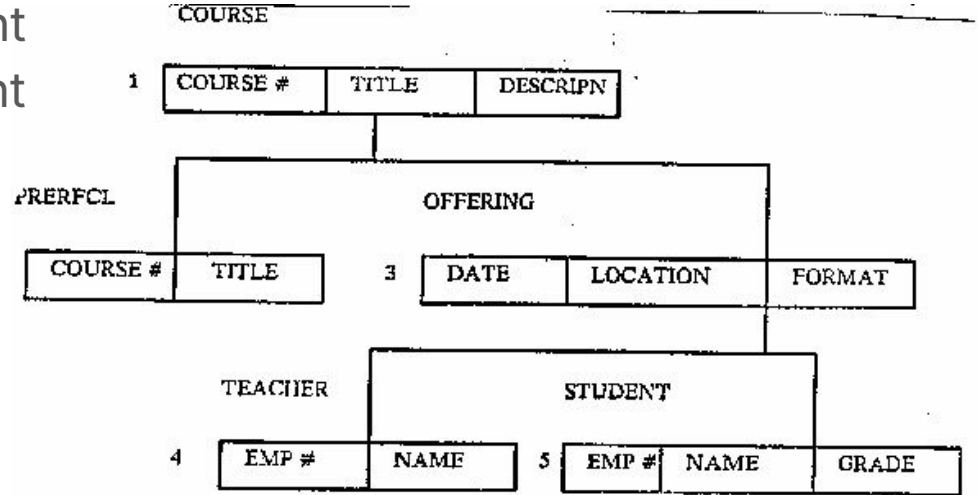
- 1960s - commercially dominant
- IBM's Information Management System (IMS)
- tree-like structure
- all child records connect to *one* parent record

*can you predict
some problems?*



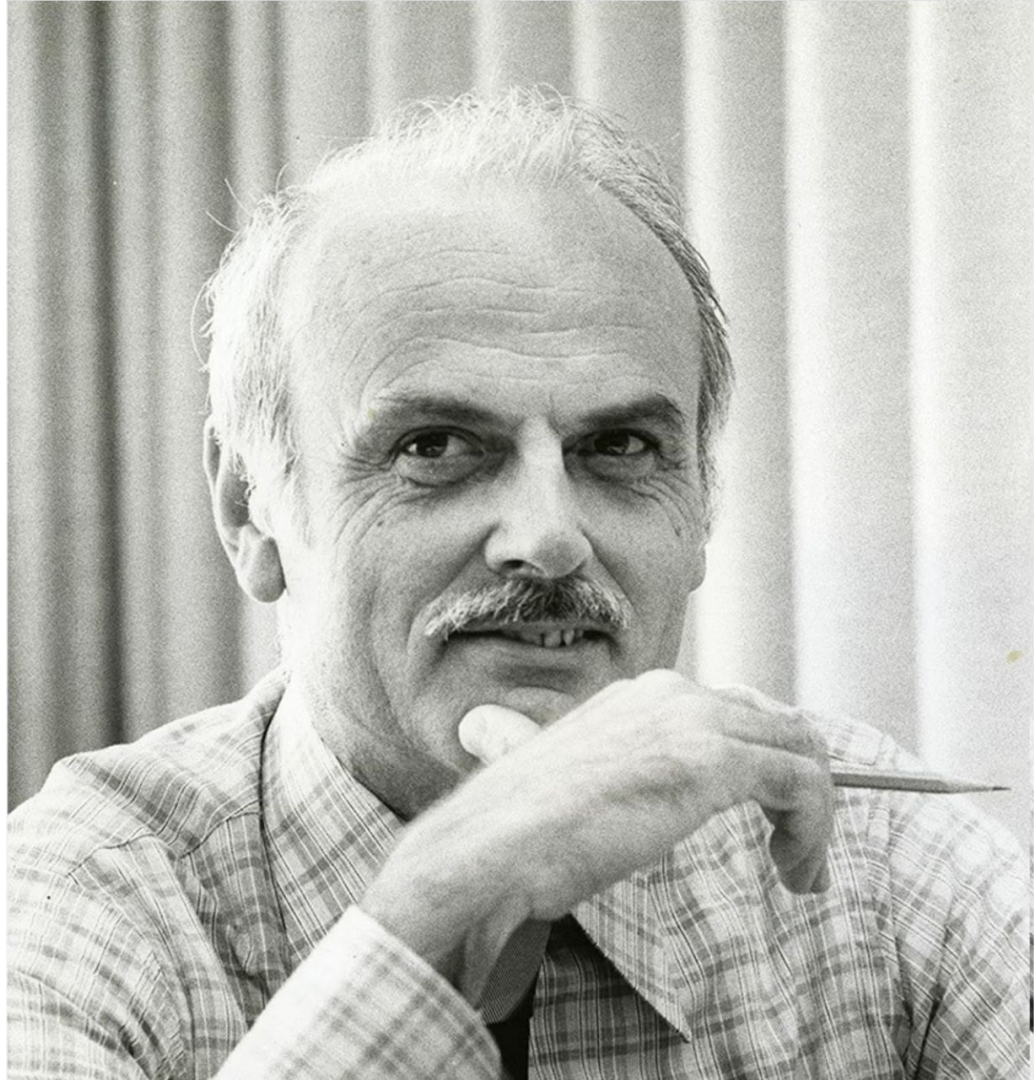
Hierarchical Databases

- 1960s - commercially dominant
- IBM's Information Management System (IMS)
- tree-like structure
- all child records connect to *one* parent record
- what if one offering removed?
- what if one teacher multiple offerings?
- what if one student enrolled in multiple courses?



Relational Databases

- proposed by E. F. Codd
- way to address M:N relationships
- based on *relational theory* from mathematics
- relation: association btw sets
- use the natural structure of the data only, no additional structure imposed
- maximal independence between data and programs



Relational Databases

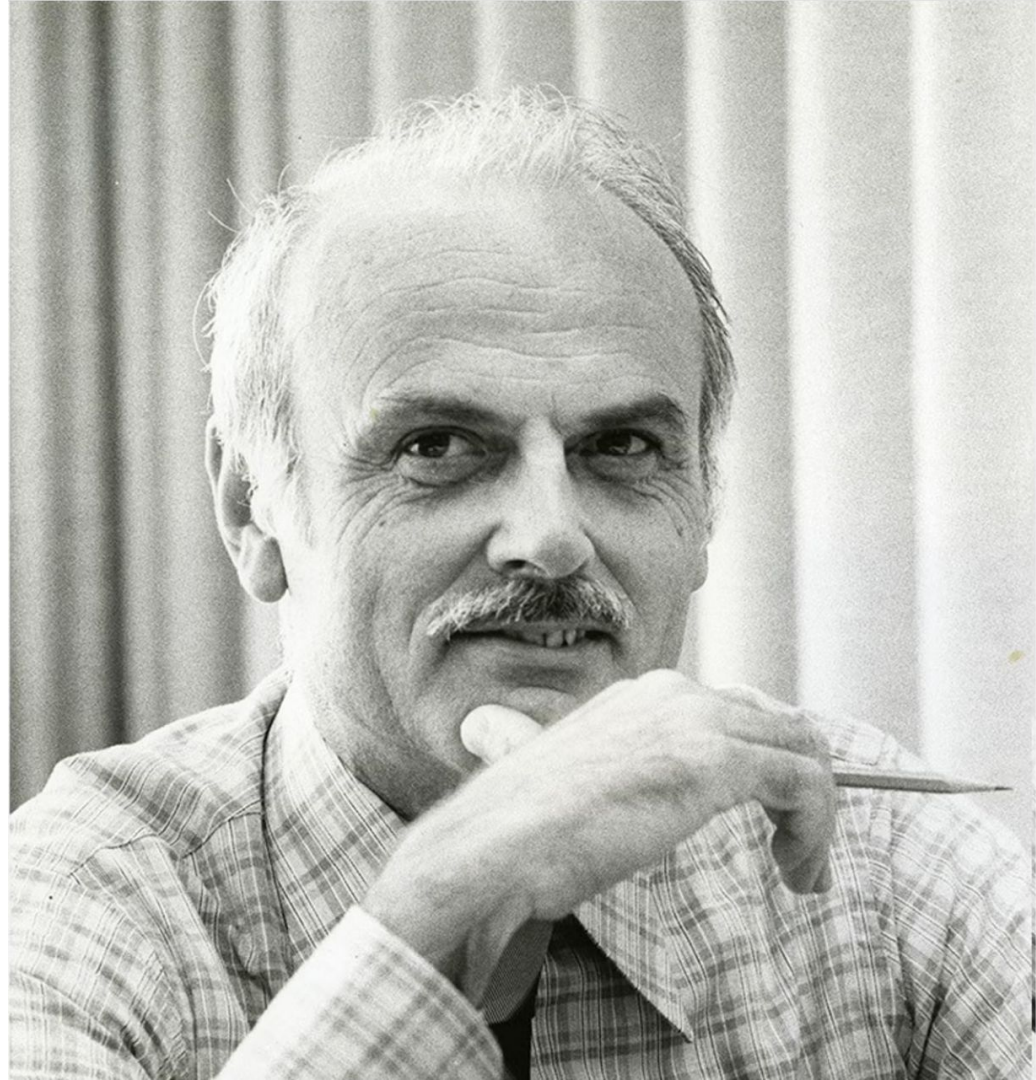
Information Retrieval

A Relational Model of Data for Large Shared Data Banks

E. F. CODD

IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.



Relational Databases

- store data in tables
- think of organization like a spreadsheet (rows, columns)

Relational Databases

- store data in tables
- think of organization like a spreadsheet (rows, columns)

each row represents one data point



Student_ID	First_Name	Last_Name	Assignment_1_Submission_Time
A1453029	Raksha	Jha	1976-01-14 23:52:12
A1822395	James	Brown	1976-01-14 17:04:43
A1108273	Yan	Zhang	1976-01-12 12:35:27

Relational Databases

- store data in tables
- think of organization like a spreadsheet (rows, columns)

 each column is a type of information stored

Student_ID	First_Name	Last_Name	Assignment_1_Submission_Time
A1453029	Raksha	Jha	1976-01-14 23:52:12
A1822395	James	Brown	1976-01-14 17:04:43
A1108273	Yan	Zhang	1976-01-12 12:35:27

Relational Databases

- store data in tables
- think of organization like a spreadsheet (rows, columns)

 **attributes**

Student_ID	First_Name	Last_Name	Assignment_1_Submission_Time
A1453029	Raksha	Jha	1976-01-14 23:52:12
A1822395	James	Brown	1976-01-14 17:04:43
A1108273	Yan	Zhang	1976-01-12 12:35:27

Relational Databases

- store data in tables
- think of organization like a spreadsheet (rows, columns)
- **attributes:** properties or characteristics of a specific database entity (aka column or field)

*what are some possible attributes
for a customer table?*

Relational Databases

- store data in tables
- think of organization like a spreadsheet (rows, columns)
- **attributes:** properties or characteristics of a specific database entity (aka column or field)

LOTS! email, phone number, name, address, demographic attributes like age/DOB, gender, income, behavioral data like website activity (page visits, time on site, items added to cart), email activity (open rates, click through rates, unsubscribes), transaction data like products purchased, transaction dates, order value, payment method, returns/refunds, review/rating scores, inferred data like interests

Relational Databases

- store data in tables
- think of organization like a spreadsheet (rows, columns)
- **attributes:** properties or characteristics of a specific database entity (aka column or field)
- table specifies attributes and data type for each attribute
- one database typically contains multiple tables
- tables linked by related attributes

Database Schema

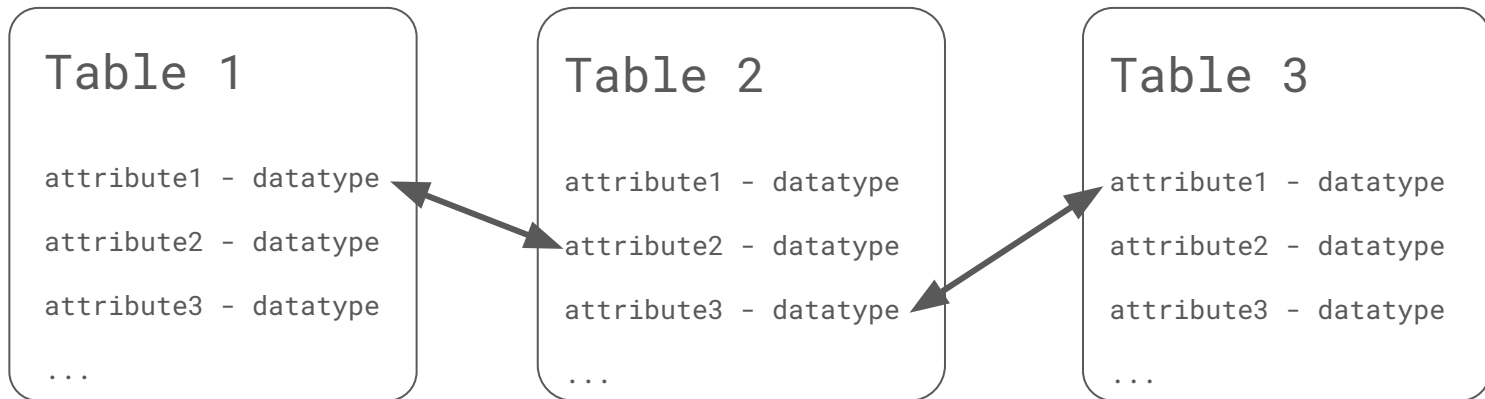
- structural blueprint
- describes how data is organized, stored, related

Database Schema

- structural blueprint
- describes how data is organized, stored, related
- Step 1: Decide which tables you need
- Step 2: For each table, decide which attributes & data types you need
- Step 3: Decide how tables are linked (are there related attributes?)

Database Schema

- structural blueprint
- describes how data is organized, stored, related
- Step 1: Decide which tables you need
- Step 2: For each table, decide which attributes & data types you need
- Step 3: Decide how tables are linked (are there related attributes?)



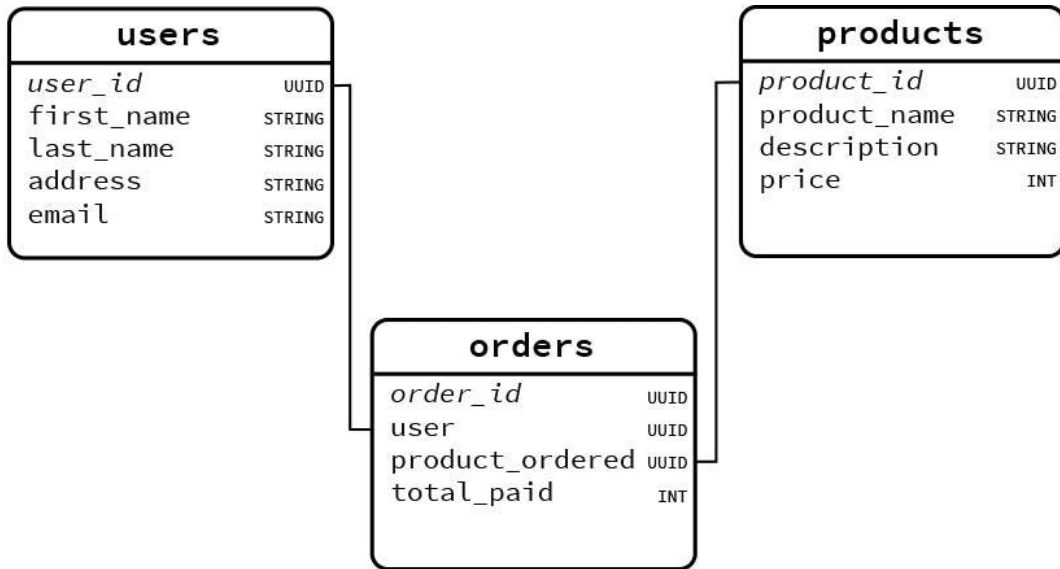
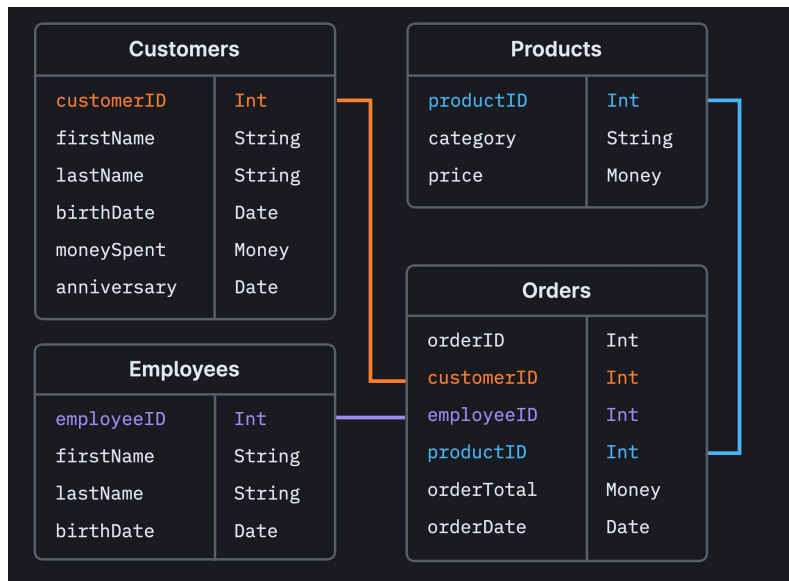
Database Schema

- structural blueprint
- describes how data is organized, stored, related
- Step 1: Decide which tables you need
- Step 2: For each table, decide which attributes & data types you need
- Step 3: Decide how tables are linked (are there related attributes?)

draft a database schema Apple could use to track its products and sales

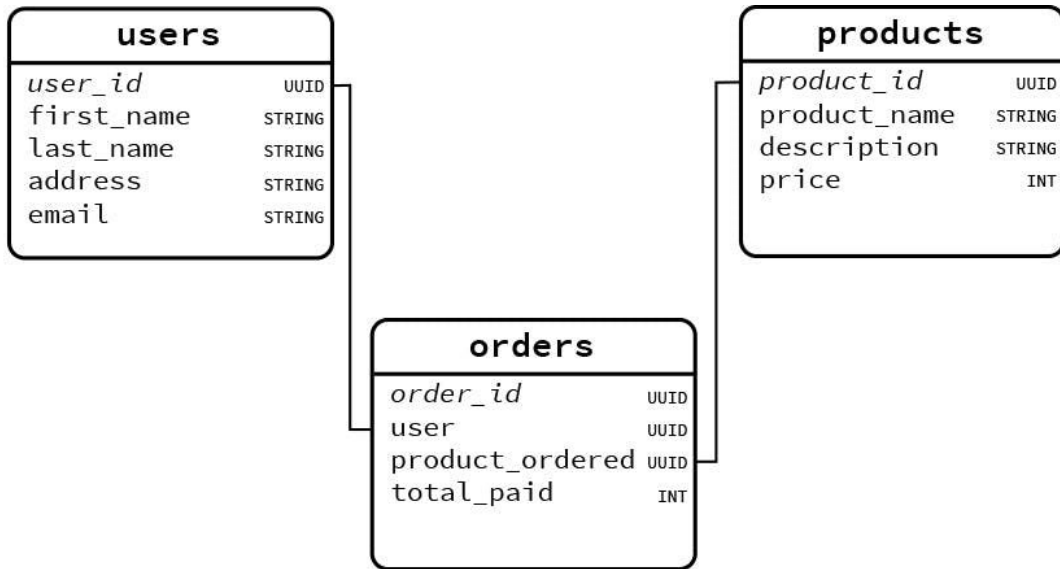
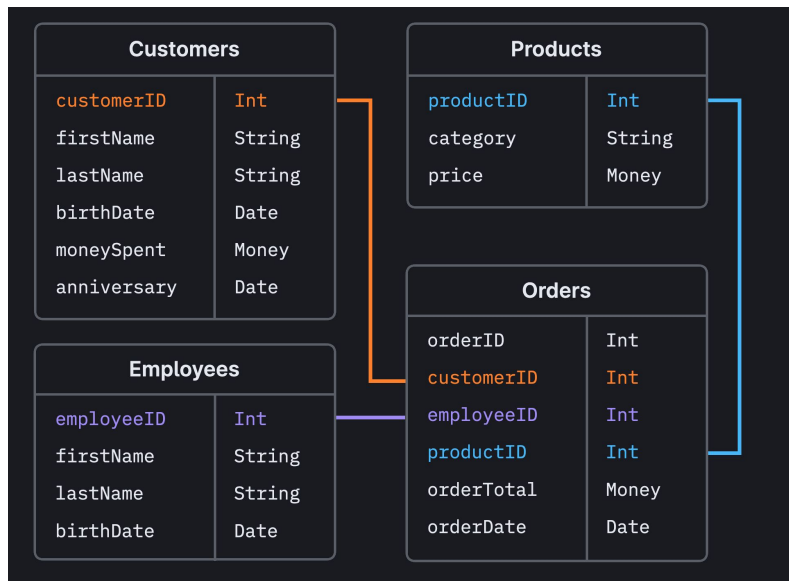
Database Schema

- structural blueprint
- describes how data is organized, stored, related



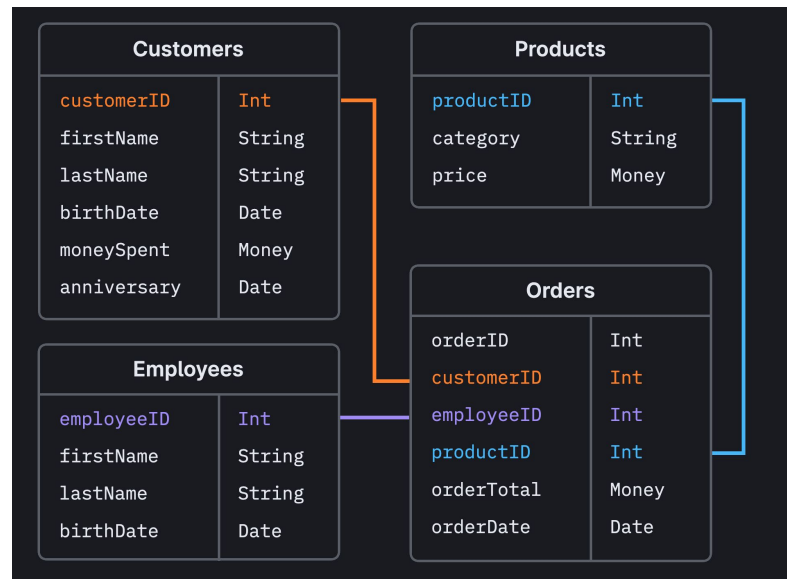
Database Schema

- structural blueprint
- describes how data is organized, stored, related
- **always start with a schema!**



Keys

- **Primary Key:** column chosen to uniquely identify every row in a table
- often a simple integer that increments as rows are added
 - tradeoffs for attempting to create "informative" primary keys
- cannot contain NULL, duplicate values
- if no primary key, can use a set of two or more columns to uniquely identify
- your life will be easier w/ primary key
- **Foreign Key:** column in one table that links to the primary key of another
- If you set foreign key, any inserted values *must exist* in parent table



Database Schema vs. Database State

- **Database Schema (or intension):** descriptions of the database structure, data types, and the constraints on the database
- **Database State (or extension):** actual data stored in a database at given point in time (also called a snapshot or occurrence)
 - Valid state: any state that satisfied the structures and constraints of the DB

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Figure 2.1

Schema diagram for the database in Figure 1.2.

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

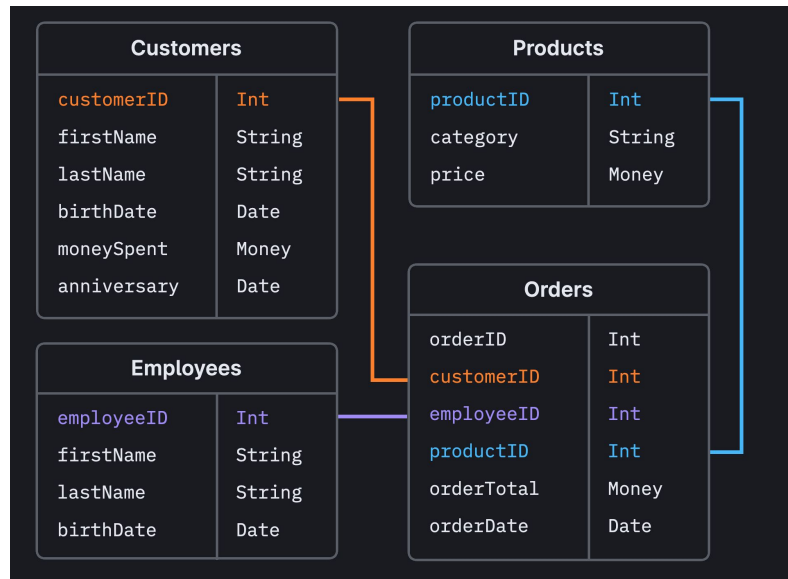
SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth

customerID	firstName	lastName	moneySpent
1	Alice	Smith	\$150.00
2	Bob	Jones	\$75.50
1	Charlie	Brown	\$300.25
4	Diana	Prince	"FREE"

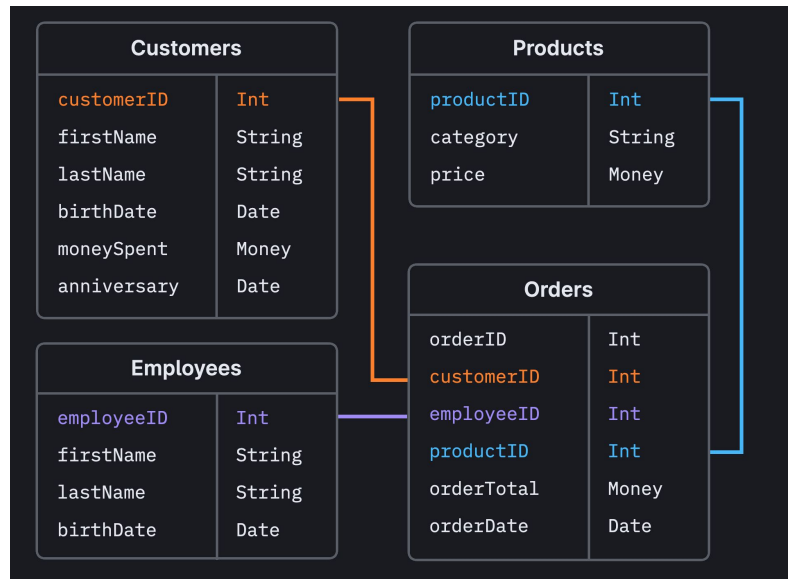
*is this a valid state
for this schema?*

orderID	customerID	orderTotal
0001	2	\$20
0002	CharlieBrown	\$75.50
0003	4000	\$28



customerID	firstName	lastName	moneySpent
1	Alice	Smith	\$150.00
2	Bob	Jones	\$75.50
1	Charlie	Brown	\$300.25
4	Diana	Prince	"FREE"

orderID	customerID	orderTotal
0001	2	\$20
0002	CharlieBrown	\$75.50
0003	4000	\$28



Benefits of Relational Databases

- **Abstraction:** data model used to hide storage details from users, programs refer to data model constructs rather than data storage details
- **Concurrency control:** allows **concurrent users** to retrieve from & update database, where each transaction is correctly executed or aborted
 - **Online Transaction Processing (OLTP):** allows hundreds of concurrent transactions per second
- **Multiple views:** different users (e.g., end user, analyst, admin) can be provided different views of the data

Structured Query Language (SQL)

- primary programming language used for relational databases
- declarative language: specify outcome rather than process

CRUD (Create, Read, Update, Delete)

- four functions that will dictate almost any data interaction
- consider the HTTP methods you learned last time

CRUD (**Create**, Read, Update, Delete)

- First step is always to create a database and tables within it

```
CREATE DATABASE database_name;
```

```
CREATE TABLE table_name(  
    column1 col1_datatype,  
    column2 col2_datatype );
```

- should have an index (we'll come back to this)

CRUD (Create, Read, Update, Delete)

- Create a database and tables within it

```
CREATE DATABASE database_name;
```

```
CREATE TABLE table_name(  
    column1 col1_datatype,  
    column2 col2_datatype );
```

- should have an index (we'll come back to this)

Products	
productID	Int
category	String
price	Money

how create this table?

CRUD (Create, Read, Update, Delete)

- Create a database and tables within it

```
CREATE DATABASE database_name;
```

```
CREATE TABLE table_name(  
    column1 col1_datatype,  
    column2 col2_datatype );
```

- should have an index (we'll come back to this)

Products	
productID	Int
category	String
price	Money

```
CREATE TABLE Products(  
    productID INT PRIMARY KEY,  
    category TEXT,  
    price MONEY);
```

CRUD (**Create**, Read, Update, Delete)

- Create a database and tables within it
- Add data to the tables
- You can add a single row using INSERT INTO:

```
INSERT INTO table(col1, col2) VALUES ('xyz', 'abc');
```

CRUD (**Create**, Read, Update, Delete)

- Create a database and tables within it
- Add data to the tables
- You can add a single row using `INSERT INTO`:

```
INSERT INTO table(col1, col2) VALUES ('xyz', 'abc');
```

- Technically you do not need to specify columns
- ⚠ In that case, you **MUST** provide a value for every single column, the exact order the columns were defined when the table was created

CRUD (**C**reate, Read, Update, Delete)

- Create a database and tables within it
- Add data to the tables
- You can add a single row using `INSERT INTO`
- You can bulk upload:

```
COPY table_name FROM '/path/to/file.csv'  
    DELIMITER ',' ;
```

- Unlike most SQL, bulk uploads are vendor-specific extensions
- This will work for PostgreSQL

CRUD (Create, **Read**, Update, Delete)

- Reading from your table will be your most common operation
- The core query is **SELECT**
- To get all the data in a table:

```
SELECT * FROM table_name;
```

- To narrow down the output, to only specific columns:

```
SELECT column FROM table_name;
```

```
SELECT column1, column 2 FROM table_name;
```

CRUD (Create, **Read**, Update, Delete)

- Reading from your table will be your most common operation
- The core query is SELECT
- To get all the data in a table: `SELECT * FROM table_name;`
- To get only specific columns: `SELECT column FROM table_name;`
- To filter the data according to attribute values:

```
SELECT column FROM table_name WHERE condition;
```

CRUD (Create, **Read**, Update, Delete)

- To filter the data according to attribute values:

```
SELECT column FROM table_name WHERE condition;
```

Condition can be:

1. Comparison (=, !=, <, >, >=, <=):

```
SELECT pid FROM products WHERE price < 100;
```

2. Logical (AND, OR, NOT):

```
SELECT pid FROM products WHERE price < 100 AND stock > 10;
```

3. Range (BETWEEN, IN):

```
SELECT pid FROM products WHERE pname IN ('Lego', 'Barbie');
```

4. Pattern-matching (LIKE, IS NULL, IS NOT NULL):

```
SELECT pid FROM products WHERE pname LIKE 'Marvel%';
```

CRUD (Create, **Read**, Update, Delete)

Some other useful tips:

- To return only unique rows:
`SELECT DISTINCT column FROM table_name;`
- To return only the first N values:
`SELECT column FROM table_name LIMIT N;`
- SQL provides many built-in math operations (SUM, ROUND, EXP, RAND):
`SELECT SUM(column) FROM table_name;`

CRUD (Create, **Read**, Update, Delete)

Some other useful tips:

- To return only unique rows:
`SELECT DISTINCT column FROM table_name;`
- To return only the first N values:
`SELECT column FROM table_name LIMIT N;`
- SQL provides many built-in math operations (SUM, ROUND, EXP, RAND):
`SELECT SUM(column) FROM table_name;`
- ⚠ Aggregate functions like SUM, COUNT cannot be used inside WHERE

CRUD (Create, Read, Update, Delete)

- To filter the data according to attribute values:

`SELECT column FROM table_name WHERE condition;`

how find which users spent at least \$100?

products	
product_id	UUID
product_name	STRING
description	STRING
price	INT

orders	
order_id	UUID
user	UUID
product_ordered	UUID
total_paid	INT

Try it out!

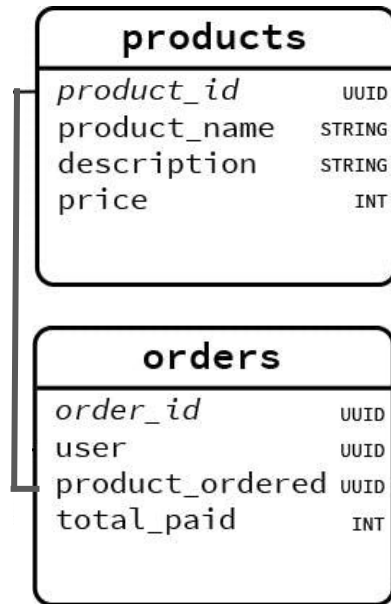


<https://sqlfiddle.com/postgresql/online-compiler?id=99fe1045-3f8e-4457-85af-6243ba552010>

CRUD (Create, Read, Update, Delete)

how find which users spent at least \$100?

- To find users who spent \$100 on a single order
`SELECT user FROM orders WHERE total_paid>100;`
- To find users who spent \$100 across all orders:
`You don't know how to do this yet`



CRUD (Create, Read, **Update**, Delete)

- To change a values:

```
UPDATE table_name  
    SET column1 = value1, column2 = value2  
    WHERE condition;
```

- Always recommended to SELECT exact same rows before UPDATE

CRUD (Create, Read, **Update**, Delete)

- To change a values:

```
UPDATE table_name  
    SET column1 = value1, column2 = value2  
    WHERE condition;
```

- Always recommended to SELECT exact same rows before UPDATE
- ⚠ If you forget the WHERE clause, ALL ROWS -- every single row -- will be overwritten with the new value

CRUD (Create, Read, **Update**, Delete)

```
UPDATE table_name  
    SET column1 = value1, column2 = value2  
    WHERE condition;
```

Products	
productID	Int
category	String
price	Money

*how update price from \$100
to \$110 for product with ID
1011*

CRUD (Create, Read, **Update**, Delete)

```
UPDATE table_name  
    SET column1 = value1, column2 = value2  
    WHERE condition;
```

Products	
productID	Int
category	String
price	Money

```
UPDATE Products  
    SET price = 110  
    WHERE productID = 1011;
```

CRUD (Create, Read, **Update**, Delete)

```
UPDATE table_name  
    SET column1 = value1, column2 = value2  
    WHERE condition;
```

Products	
productID	Int
category	String
price	Money

```
UPDATE Products  
    SET price = 110  
    WHERE productID = 1011;
```

Actually any
of these
would work!

← '\$100.00'
← '-50.25'
← '99'
110
150.50

CRUD (Create, Read, Update, **Delete**)

- To delete an entry/entries:

```
DELETE FROM table_name WHERE condition;
```

- To delete all data in a table:

```
DELETE FROM table_name;
```

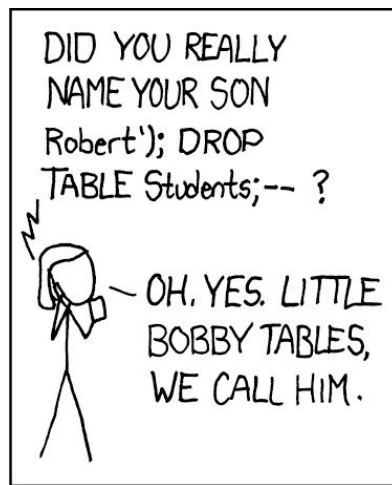
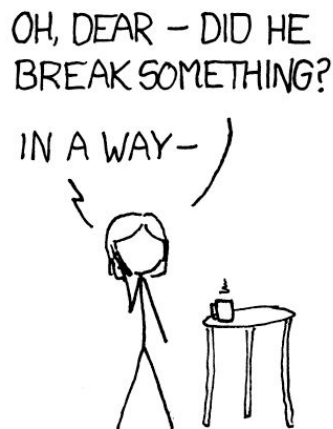
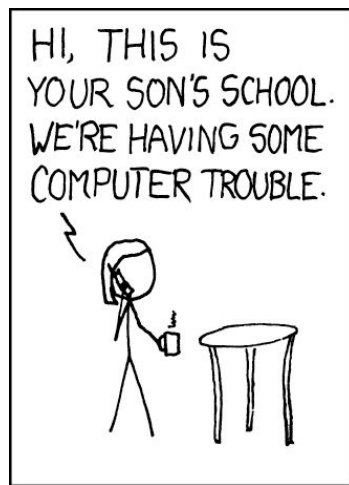
- To delete the table:

```
DROP TABLE table_name;
```

CRUD (Create, Read, Update, **Delete**)

- To delete the table:

```
DROP TABLE table_name;
```



Why all the YELLING?

- when SQL was developed in 1974, most keyboards did not have lowercase letters
- kept for syntax highlighting
- **bouma** - shape of a word
- bouma of ALLCAPS is always a rectangle
- easy to distinguish SQL QUERY vs the data



JOINS

- allows you to retrieve data from two or more tables
- all tables must share a related column

JOINS

- allows you to retrieve data from two or more tables
- all tables must share a related column

```
SELECT table_name_1.column1  
      FROM table_name_1  
      JOIN table_name_2  
      ON table_name_1.column1=table_name_2.column2 ;
```

JOINS

- allows you to retrieve data from two or more tables
- all tables must share a related column

```
SELECT table_name_1.column1  
      FROM table_name_1  
      JOIN table_name_2  
      ON table_name_1.column1=table_name_2.column2 ;
```

- if there is no related column, you cannot join those two tables

JOINS

- allows you to retrieve data from two or more tables
- all tables must share a related column

```
SELECT table_name_1.column1  
      FROM table_name_1  
      JOIN table_name_2  
      ON table_name_1.column1=table_name_2.column2 ;
```

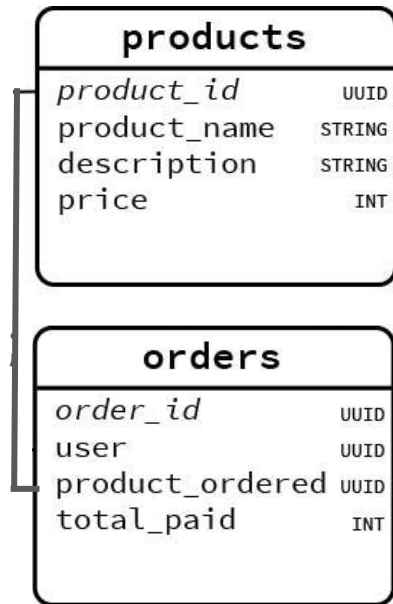
- if there is no related column, you cannot join those two tables
- ⚠ the related column is not *required* to be a Foreign Key, but often faster
- ex: joining customers and orders tables based on customer email address even though no official relationship/key

JOINS

- allows you to retrieve data from two or more tables
- all tables must share a related column

```
SELECT table_name_1.column1
      FROM table_name_1
      JOIN table_name_2
      ON table_name_1.column1=table_name_2.col2
```

*how find the name of any product
sold at least once?*



Try it out!



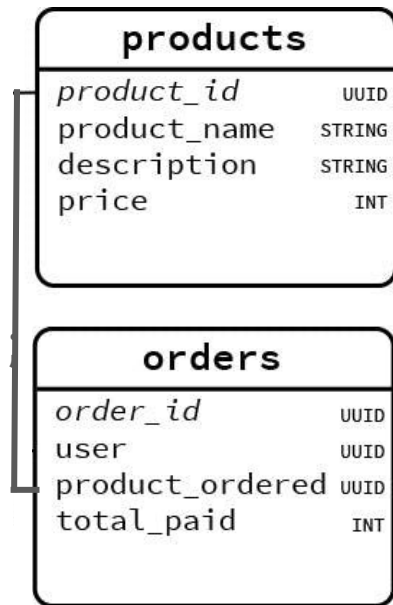
<https://sqlfiddle.com/postgresql/online-compiler?id=99fe1045-3f8e-4457-85af-6243ba552010>

JOINS

- allows you to retrieve data from two or more tables
- all tables must share a related column

```
SELECT table_name_1.column1
      FROM table_name_1
      JOIN table_name_2
      ON table_name_1.column1=table_name_2.col2
```

```
SELECT DISTINCT products.product_name
      FROM products
      JOIN orders
      ON products.product_id = orders.product_ordered;
```



JOINS

- ```
SELECT table_name_1.column1
FROM table_name_1
JOIN table_name_2
ON table_name_1.column1=table_name_2.column2 ;
```

*extra challenge:  
how find all products  
with at least \$200 in sales?*

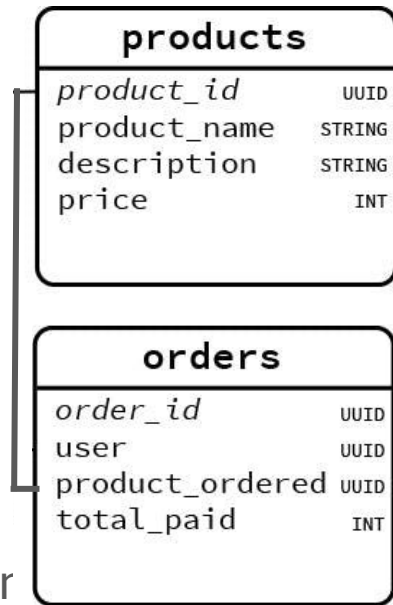
| products          |        |
|-------------------|--------|
| <i>product_id</i> | UUID   |
| product_name      | STRING |
| description       | STRING |
| price             | INT    |

| orders          |      |
|-----------------|------|
| <i>order_id</i> | UUID |
| user            | UUID |
| product_ordered | UUID |
| total_paid      | INT  |

# JOINS

- allows you to retrieve data from two or more tables
- all tables must share a related column
- ```
SELECT table_name_1.column1  
FROM table_name_1  
JOIN table_name_2  
ON table_name_1.column1=table_name_2.column2
```

```
SELECT products.product_name, SUM(orders.total_paid)  
FROM products JOIN orders  
ON products.product_id = orders.product_ordered  
GROUP BY products.product_id  
HAVING SUM(orders.total_paid)>200;
```



CRUD (Create, **Read**, Update, Delete)

- To create grouped categories:

```
SELECT AVG(column) FROM table GROUP BY column;
```

```
SELECT AVG(rating) FROM movies GROUP BY genre;
```

- Needs to return aggregate function (SUM, MAX, AVG)

- Would not know which individual row value to return otherwise
- Can also return the grouped-by attribute

- **WHERE** filters individual rows, before grouping

- **HAVING** filters aggregated groups, after grouping

```
SELECT column FROM table GROUP BY column HAVING condition;
```

CRUD (Create, Read, Update, Delete)

how find which users spent at least \$100 across all orders?

- To find users who spent \$100 on a single order
`SELECT user FROM orders WHERE total_paid>100;`
- To find users who spent \$100 across all orders:
`???`

products	
<i>product_id</i>	UUID
product_name	STRING
description	STRING
price	INT

orders	
<i>order_id</i>	UUID
user	UUID
product_ordered	UUID
total_paid	INT

CRUD (Create, Read, Update, Delete)

how find which users spent at least \$100 across all orders?

- To find users who spent \$100 on a single order
`SELECT user FROM orders WHERE total_paid>100;`
- To find users who spent \$100 across all orders:
`SELECT user FROM orders GROUP BY user
HAVING SUM(total_paid)>100;`

products	
<i>product_id</i>	UUID
product_name	STRING
description	STRING
price	INT

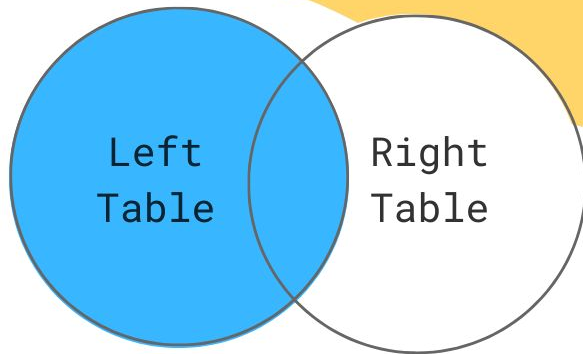
orders	
<i>order_id</i>	UUID
user	UUID
product_ordered	UUID
total_paid	INT

JOINS

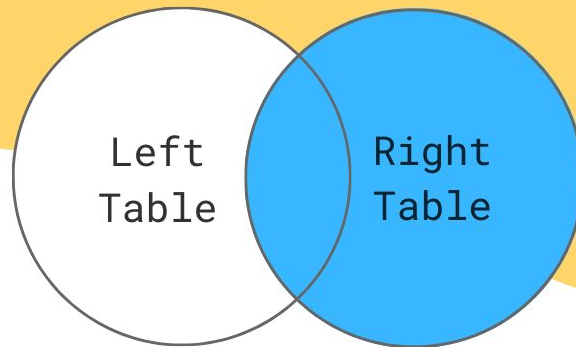
- allows you to retrieve data from two or more tables
- all tables must share a related column

- ```
SELECT table_name_1.column1
 FROM table_name_1
 JOIN table_name_2
 ON table_name_1.column1=table_name_2.column2 ;
```

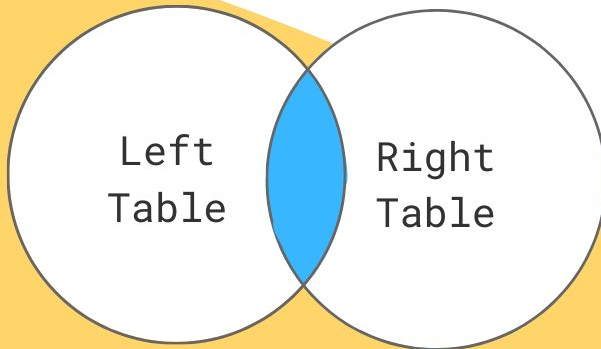
## LEFT JOIN



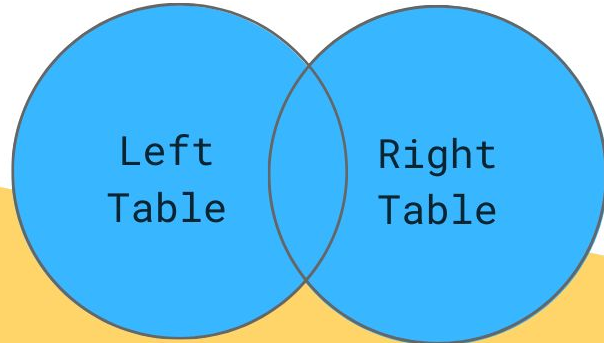
## RIGHT JOIN



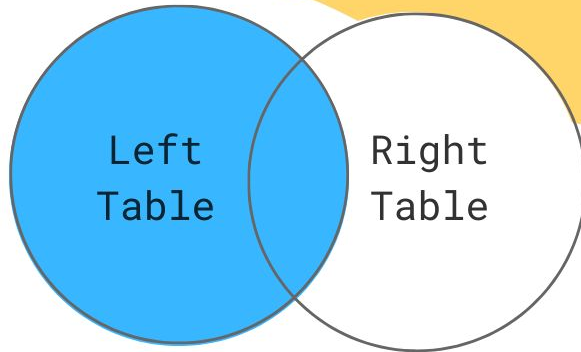
## INNER JOIN



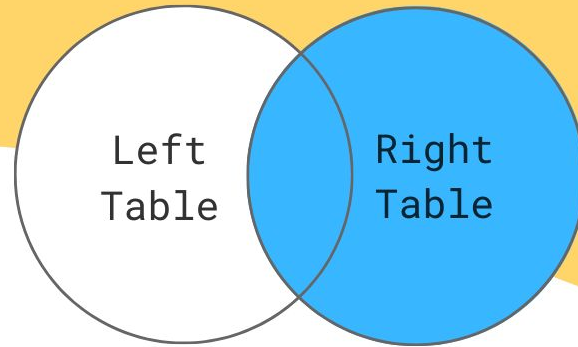
## FULL JOIN



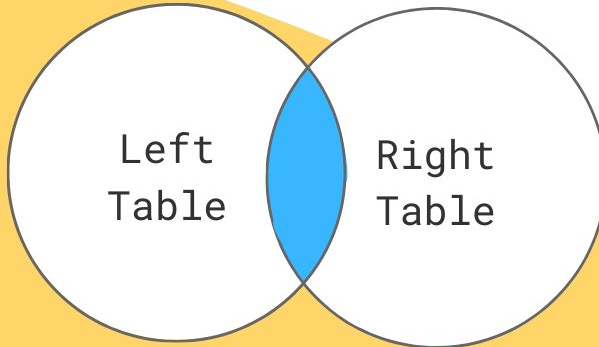
## LEFT JOIN



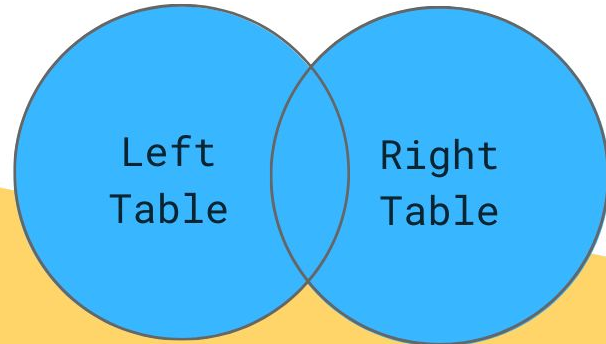
## RIGHT JOIN



## INNER JOIN



## FULL JOIN



# JOINS

- allows you to retrieve data from two or more tables
- all tables must share a related column
- ```
SELECT table_name_1.column1  
      FROM table_name_1  
      JOIN table_name_2  
      ON table_name_1.column1=table_name_2.column2 ;
```
- **Inner join:** *default*, only returns rows with an exact match in both tables
- Other joins (Outer, Left, Right, Full): useful if you want to keep data without a match in both tables

Relational Databases

- store data in tables
- think of organization like a spreadsheet (rows, columns)
- **attributes:** properties or characteristics of a specific database entity (aka column or field)

LOTS! email, phone number, name, address, demographic attributes like age/DOB, gender, income, behavioral data like website activity (page visits, time on site, items added to cart), email activity (open rates, click through rates, unsubscribes), transaction data like products purchased, transaction dates, order value, payment method, returns/refunds, review/rating scores, inferred data like interests

What Do Marketers Do With Their Data?

- **Identity Resolution:** combine scattered data points from different devices & channels to create a single profile to avoid duplicate ads/costs
- **Customer Segmentation:** divide the full audience into smaller, distinct groups based on shared traits, target segments with specific messaging, allows personalized campaigns and more focus on high value segments
- **Retargeting:** identify website visitors who left without purchasing, follow up with increased ads / outreach to turn them into paying customers
- **Dynamic Creative Optimization (DCO):** tailored ads for each user with mix & match headlines/text/images/calls to action/etc.
- **Ad Performance Tracking:** track Key Performance Indicators (KPIs) like Customer Acquisition Cost (CAC), Click Through Rate (CTR), Return on Ad Spend (ROAS) to measure cost effectiveness of ads

Types of Databases for Targeted Advertising

- **Customer Data Platforms (CDP)**: designed to unify diverse data sources
- primary goal: create a unified profile from multiple sources/channels
- For example, log and store every interaction user has with company website:
 - Every page visited
 - Search queries on site
 - Dwell time - how long user spends on specific page
 - Engagement metrics - scroll depth, cursor movements
 - Filling out forms
 - Products added to shopping cart
 - Completed purchase
 - Payment methods
 - Loyalty programs
 - Conversions (clicking from ad, clicking from email)
 - Time since most recent visit/purchase

Types of Databases for Targeted Advertising

- **Customer Data Platforms (CDP)**: designed to unify diverse data sources
- primary goal: create a unified profile from multiple sources/channels
- For example, log and store every interaction user has with company website: page visit, search logs, dwell time, engagement, cart adds, etc.
- Also includes data on: email, apps, 3rd party websites, online ads, chat
- Can also include in-store checkouts or mobile/offline POS (ex: Square)



Are These Actually Relational Databases?

- aka

standing up against big postgres

- many are, but by no means all!
- even relational databases often offer "drag and drop" interactions to marketing teams, rather than writing SQL
- some task constraints mean relational databases are not the best fit
 - there are many types of NoSQL databases: data lakes, document DBs, KV DB
 - priority is fast storage, not fast queries → data lake
 - extremely low latency needed → key value engine
- many actually still provide support for SQL-like queries

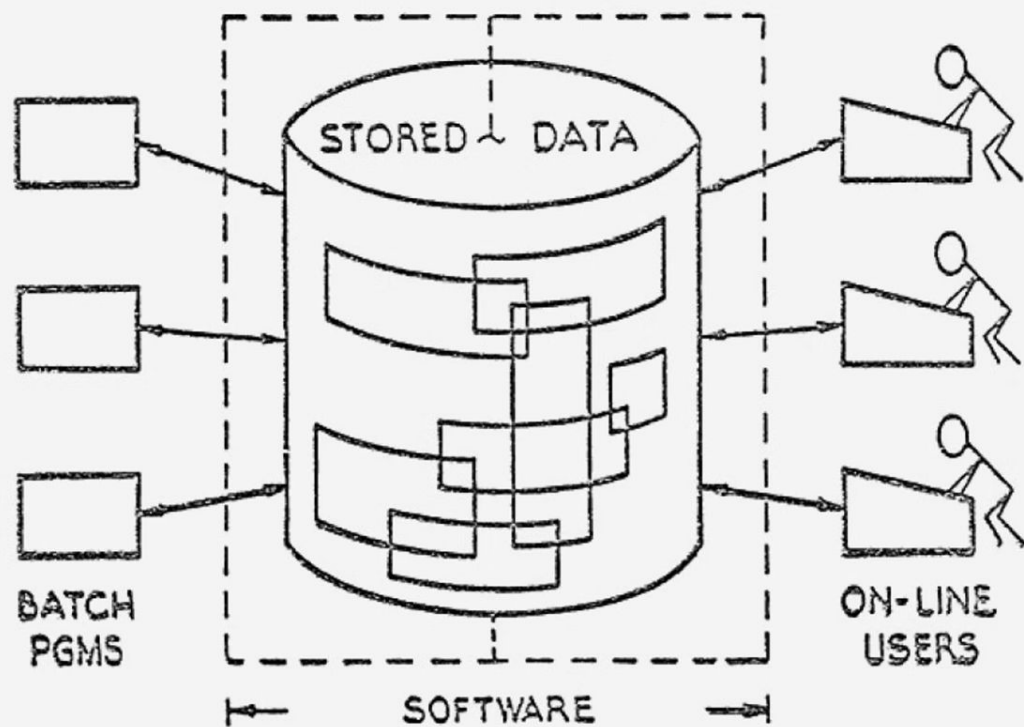
Types of Databases for Targeted Advertising

- **Data Management Platform (DMP)**: compared with CDP, more focused on anonymous (no customer profile) data, often shorter-term than CDP
primary goal: find *lookalike audiences* (we will return to this later!)
- **Customer Relationship Management (CRM) Database**: treated as "source of truth" for primarily internal customer info (ex: contact email)
primary goal: manage sales pipelines and customer relationships
- **Real-Time Data Platform (RTDP)**: provides lookups for device ID or cookie profile to place ad bids, needs sub-millisecond lookups so typically key-value engine
- **Ad Asset Libraries**: store creative assets (images, videos, call-to-action buttons) used within ads, especially components used within DCO

Challenges for Ad Targeting

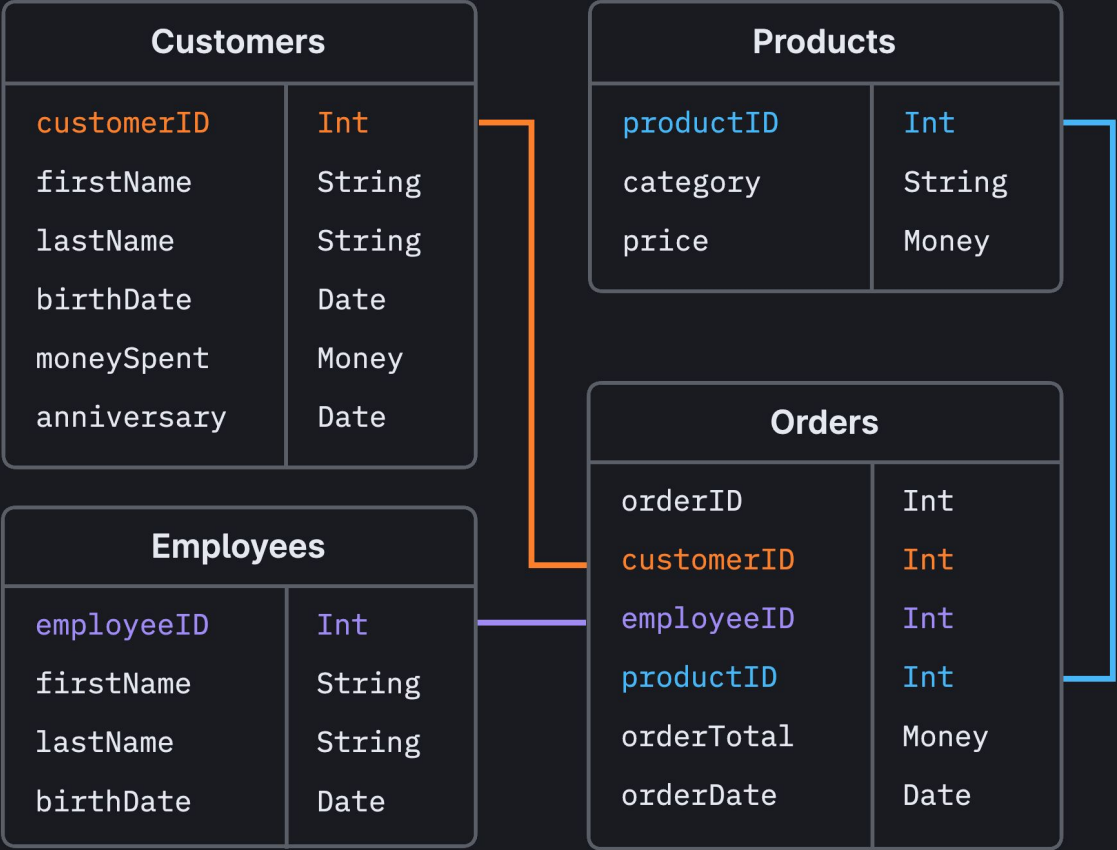
- **Extremely low latency:** The Demand-Side Platforms (DSPs) and ad servers require sub-50ms response times
 - Have you picked good indexes? Are you doing unnecessary joins?
 - May need NoSQL, reduce network overhead
- **Unstable keys:** if no authenticated data (e.g., email address) is available, customer identities are often inferred (e.g., using browser fingerprinting)
 - Values like device type, browser setup, location can change
 - Web browsers/OS have begun purging identifiers automatically
 - Ad fatigue (too many ads shown), poor measurement of ROAS
- **Regulatory compliance:** stricter privacy and consent laws (e.g., GDPR, CCPA) are being introduced, but differ by region

A DATABASE SYSTEM



one database
generally
contains
multiple
tables

each table
specifies
attributes and
datatypes



tables
linked by
related
attributes